

1 User-space Paging

Fast memory paging is essential for general-purpose “emerging applications” such as those on smartphones, where cases like app startup rely on fast fault handling to reduce startup latency for the user. *Chen_JWLLLWHLYWYPX₂4claimthatmicrokernelsareinherenttripfromthekerneltothepager”.Asecondaryclaimisthat“capabilities[...]introducesignificantperformanceoverslower than Linux”.*

Due to this, the pager has been moved inside the kernel in HongMeng, eliminating the extra round-trip IPC. The kernel pager, on a fault, directly maps a pre-allocated memory frame to the faulting page and then records an operations log for the user-space memory manager server to implement policies *Chen_JWLLLWHLYWYPX₂4.ThisdepartsfromLiedtke’sminimalityprincipleLiedtke₉5, andwebelievethat*

1.1 HongMeng benchmarks

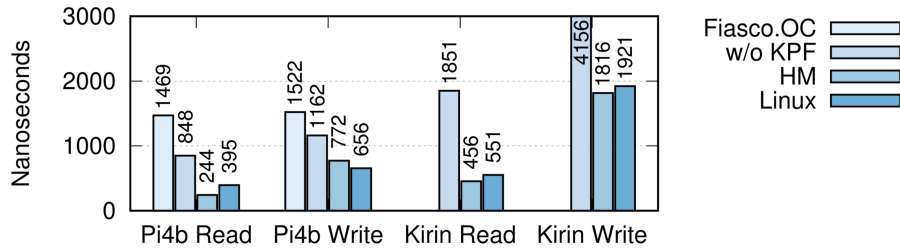


Figure 1: Page fault latency of private anonymous memory: HongMeng. w/o KPF denotes the HongMeng kernel with a user-space pager.

This benchmark involved mapping anonymous memory via *mmap* without pre-populating, then performing page-by-page operations to trigger page faults on first access. The benchmark measured average latency, calculated by triggering a batch of page faults and measuring the total elapsed time, and dividing it by the number of pages. Source: *Chen_JWLLLWHLYWYPX₂4*

?? shows the performance improvement of page fault latency that *Chen_JWLLLWHLYWYPX₂4achievedinthe*

1.2 Argument for user-space paging

The argument is that the IPC cost and frame capability invocation (from mapping) are not as significant a cause of performance degradation as claimed by *Chen_JWLLLWHLYWYPX₂4.Itshouldbepossibletoimplementperformant(similartoLinux)user-spacepagingwithoutcompromisingonaprincipledmicrokernel design.Thus, auser-spacepageronLionsOS(whichisbasedonseL4)shoulddemonstratethatprivateanonymouspagefaultlatencysimilarityPerformanceandCompatibilityintheHongMengProductionMicrokernel.”*

1.3 Our benchmarking

Minor page fault latency

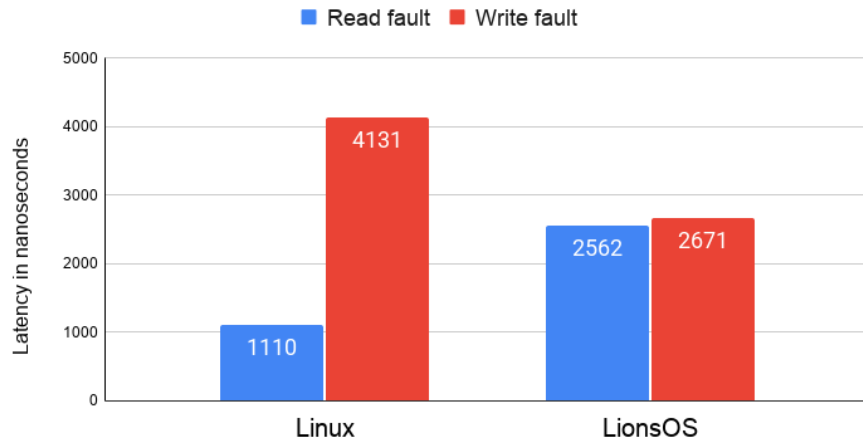


Figure 2: Page fault latency of private anonymous memory: LionsOS. Measured on Pi4b with a 1.5GHz clock rate. Linux version is 6.12.61-v8. Read is optimised with zero page.

?? shows our benchmarking of page fault latency which follows the same methodology as in ???. This benchmark shows the average page fault latency measured by triggering a batch of page faults and measuring the total elapsed time divided by the number of pages. This was run for 16384 4096 byte pages on Pi4b running at 1.5GHz clock frequency. The page faults were triggered by accessing private anonymous, not pre-populated memory, mapped via *mmap*.

Write fault

The general process to service write faults is:

1. Get a (pre-allocated) frame cap to an unused memory frame.
2. Create shadow page tables to keep track of this frame.
3. Map (pre-allocated) intermediary paging structures.
4. Map frame cap to faulting page.
5. Resume execution on faulting PD

Read fault

The *mmap()* syscall allocates zero-filled pages, hence the global zero-page optimisation is triggered for Linux, requiring less bookkeeping as the global zero

page can simply be read-only mapped. The LionsOS pager also implements the same optimisation, mapping a global zero frame. However, as the same frame cannot be mapped into the same vspace using the same capability, a pre-allocated capability copy must be used to do the mapping, and as we cannot lose track of the capability (as we do not want to leak CNode slots), we must also create shadow page tables for this capability copy and map this frame to the faulting page. This results in a similar amount of bookkeeping as write faults and the same number of kernel invocations (mapping intermediary paging structures and page mapping) resulting in a similar latency. ?? shows that this affects Fiasco.OC similarly as they also use a similar capability model and therefore read and write fault latencies are not significantly different.

?? shows an approximate average latency of 2600ns per fault for both write and read faults. The `PAGE_MAP` operation accounts for approximately 847-885ns (read, write). Approximately 200-300ns are used for bookkeeping and the rest is accounted for by the fault delivery and round trip IPC. The pager is implemented as a passive server to reduce scheduling overhead.

1.4 Conclusion

We observe from ?? that write fault handling can be done with a lower latency than Linux, despite the inclusion of the extra round-trip IPC and capability operations. The global zero-page optimisation is approximately $2.5\times$ slower than Linux, but this is an edge case optimisation that only occurs on reads to zeroed out pages or previously unaccessed pages. The slowdown cited by Chen,*JWLLLWHLYWYPX₂4isnotassignificantforthereadcaseandforthewritecase,LionsOSachievesbetter*

This shows that the argument for introducing the pager into the kernel to achieve better performance is not so obvious. In order to keep the option of functional verification for the kernel open, the pager should be implemented as a user-space server.

1.5 Weird results from ??

I failed to match the measurements from Chen,*JWLLLWHLYWYPX₂4,andherearesomeweirdthingsIhaveno*
 Kirin 9000 (little core) which is a cortex a55 at 2.05GHz is drastically slower than Pi4b which is a cortex a73 at 1.5GHz.

Fiasco.OC achieves around 1500ns latency per fault. This is 2250 cycles to do the IPC and fault delivery, mapping operations and bookkeeping. Given that their fastpath IPC is slower than seL4's, this seems implausible.

Chen,*JWLLLWHLYWYPX₂4claimsforseL4that"theround-trip(tothepager)offaulthandlingtakesa*
tripIPCfastpathonseL4onArmv8atakesaround1000cycles,notthe210cyclesclaimedhere.